GENERATIVE MODELS

Kourosh GERAYELI

CEDRE 2026

INTRODUCTION

What's the difference between generative models and discriminative models?

- discriminative models learn boundaries that separate different classes, while generative models learn the distribution of different classes.

we will consider generative models that solve these tasks:

- Noise to image (DCGAN)
- Text to image (diffusion models)
- Image to image (StyleGAN, cycleGAN, diffusion models)

EVALUATION OF GENERATIVE MODELS IN COMPUTER VISION

- FID (Fréchet Inception Distance) :

"How similar are the fake images to the real ones?"

- Low FID → Good (fake images look real)
- High FID → Bad (fake images look different from real ones)

Generally, it is really hard to come up with meaningful metrics for evaluating generative models. Because often you don't have a solid "ground truth", and it is difficult to quantify the quality of an image. FID is the most commonly used metric, but it is not perfect.

🖼 Real images (from your dataset)
🎨 Fake images (generated by your model)

FID gives you **a number** that measures how close the generated images are to real images.

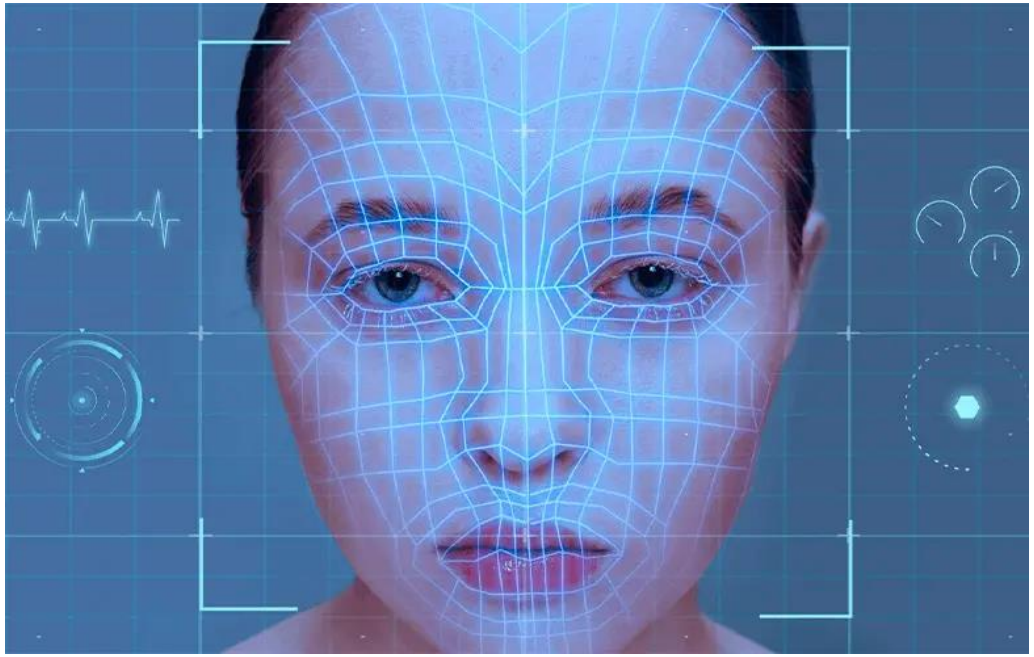
Core Idea of FID:

Instead of comparing images pixel by pixel (which doesn't work well), FID does something smarter:

- It passes both real and fake images through a pretrained network called Inception-v3
- It extracts high-level features (not raw pixels!)
- It compares the statistics of those features

So FID compares:

- "Do the fake images look statistically similar to real images in feature space?"



Extract Features

- Both real and generated images are passed through **Inception-v3**.
- Instead of using the final classification output, we take features from an internal layer (usually 2048-dimensional vector).

So now:

- Each image becomes a **2048-number feature vector**
- These vectors describe high-level content like shapes, textures, objects

- **Build Two Distributions**

Now you have:

- Many feature vectors for real images
- Many feature vectors for fake images

For each group, we compute:

- Mean (μ)
- Covariance (Σ)

So we summarize each group as a Gaussian distribution:

- Real images → (μ_real, Σ_real)
  Fake images → (μ_fake, Σ_fake)

➢ Now we measure how far these two Gaussian distributions are using something called the **Fréchet distance**.

INCEPTION-V3

- A deep CNN trained on ImageNet
- Very good at understanding images
- Knows about objects, textures, patterns, shapes

- **What Are “Features”?**

When an image goes through Inception-v3:

- Early layers detect edges
- Middle layers detect shapes
- Deep layers detect objects & structure

FID uses the activations from a deep layer before classification.

These activations:

- Are 2048 numbers
- Represent high-level image content
- Ignore small pixel noise
- Capture semantic meaning

Think of them as:

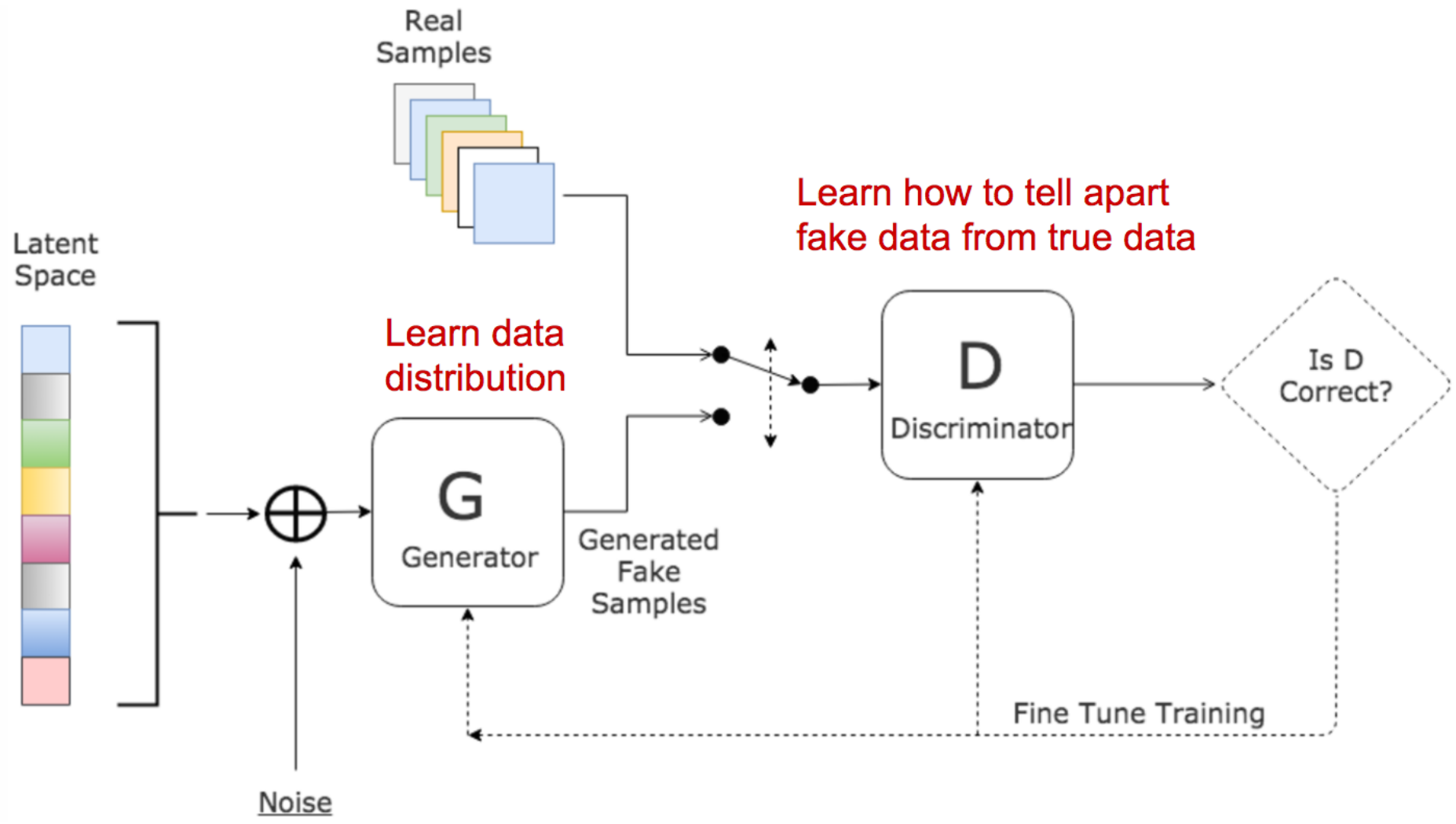
A compact description of what the image “contains”,

Instead of comparing pixels, we compare these semantic descriptions.

OTHER METRICS

- PSNR (peak signal-to-noise ratio)
- SSIM (Structural Similarity Index) is a metric in the range [0, 1] where 1 is a perfect match.
- Inception score, it is calculated using the features on the inceptionv3 model. The higher the better.
- CLIP Score, this metric was introduced in CLIPSCORE is used to evaluate the quality of text to image models.

GAN (GENERATIVE ADVERSARIAL NETWORKS)



Generative Adversarial Networks (GANs) are a class of deep learning models introduced by Ian Goodfellow and his colleagues in 2014. The core idea behind GANs is to train a generator network to produce data that is indistinguishable from real data, while simultaneously training a discriminator network to differentiate between real and generated data.

• **Architecture overview:** GANs consist of two main components: the generator and the discriminator.
• **Generator:** The generator takes random noise \(z\) as input and generates synthetic data samples. Its goal is to create data that is realistic enough to deceive the discriminator.
• **Discriminator:** The discriminator, akin to a detective, evaluates whether a given sample is real (from the actual dataset) or fake (generated by the generator). Its objective is to become increasingly accurate in distinguishing between real and generated samples.
A common analogy that can be found online is that of an art forger/painter (the generator) which tries to forge paintings and an art investigator/critic (the discriminator) which tries to detect limitations.

---

latent space:
. Definition
•The **latent space** is a high-dimensional vector space where each point corresponds to a possible configuration that the GAN can generate.
•Each vector (point) in this space is usually randomly sampled (often from a Gaussian or uniform distribution) and is fed into the **generator** network.
•The generator learns to map these latent vectors into realistic images.
Think of latent space as a “blueprint” of all possible images the GAN can produce. The coordinates in this space encode abstract features like shape, color, style, or orientation, even though they are not directly interpretable as pixels.

1. Generator (G)

- Input: random noise vector z~p(z) (usually Gaussian or uniform)
- Output: fake data sample G(z)

2. Discriminator (D)

- Input: image (real or fake)
- Output: probability that input is real
- D(x) = P(real|x)

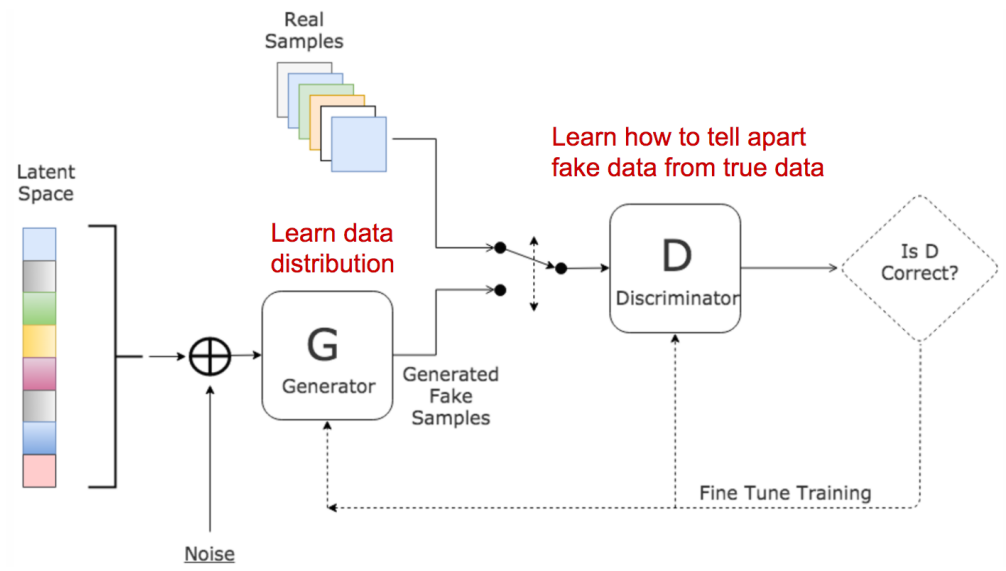
So:

•D(x_real) → close to 1
•D(G(z)) → close to 0 (initially)

3. The Game Theory Perspective

- GANs are based on **minimax game theory:** Generator wants to minimize:
- $\log(1 - \mathrm{D}(\mathrm{G}(\mathrm{z})))$
- Discriminator wants to maximize :

$$\log D(x) + log(1 - D(G(z)))$$



Intuition

- D tries to correctly classify real vs fake.
- G tries to fool D.
- Over time, G improves.
- If training succeeds → D cannot distinguish real from fake → outputs 0.5 everywhere.
- At equilibrium
  - $p_{model}(x) = p_{data}(x)$

Instead of predicting labels (classification) or numbers (regression), they **learn to generate data** that looks like the training distribution.
If you train a GAN on:
•Face images → it generates new faces
•MRI scans → it generates synthetic MRIs
•Microscopic images → it generates synthetic particles
•Paintings → it generates art
The key idea:
Instead of explicitly modeling the probability distribution p(x), we learn to sample from it.

D(xreal)→1 → if the image is real, D should predict high probability.
D(G(z))→0D(G(z)) \to 0D(G(z))→0 → if the image is fake, D should predict low probability.

Discriminator is basically a "critic" saying: *"This looks real"* or *"This looks fake."*
Initially, G(z)G(z)G(z) is random, so D(G(z))≈0D(G(z)) \approx 0D(G(z))≈0 because it's obvious the image is fake.

what does G produces at the very beginning? Why? At the start, G(z) produces nonsense because it hasn't learned yet !

NOTEBOOK 5. GAN

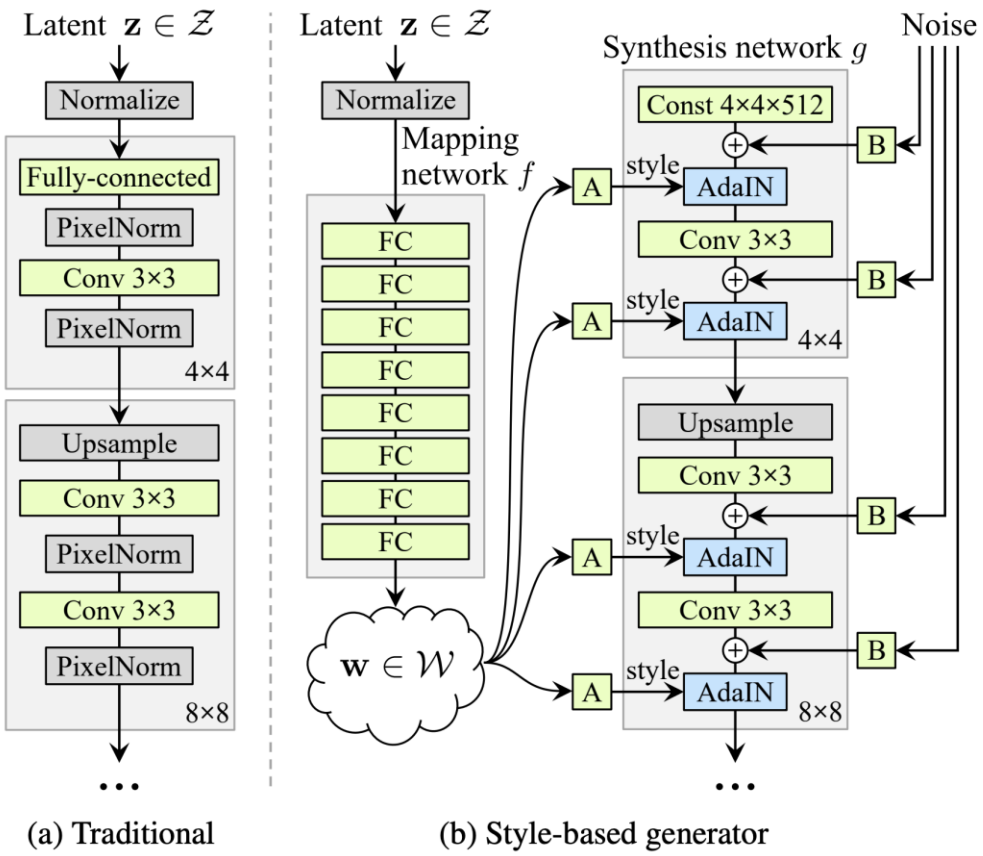
STYLEGAN

StyleGAN is an improved GAN made especially for generating high-quality images especially faces.

It was created by NVIDIA.
The key idea:
👉 Instead of generating an image directly, StyleGAN controls the image using "styles" at different levels.

Imagine you're building a face in Photoshop using layers:
- Layer 1 → Face structure
- Layer 2 → Hair
- Layer 3 → Expression
- Layer 4 → Skin texture

StyleGAN works similarly:
Each layer of the network controls a different "style" level of the image.



What does “Style” mean here?

Think about drawing a face.

There are different levels of information:

- 🧠 High-level features:
  - Face shape
  - Gender
  - Age
  - Pose
- 🧬 Medium-level features:
  - Hair style
  - Eye shape
  - Glasses
- 🎨 Fine details:
  - Skin texture
  - Wrinkles
  - Pores
  - Lighting details

StyleGAN separates these levels and controls them independently.

That’s the big innovation.

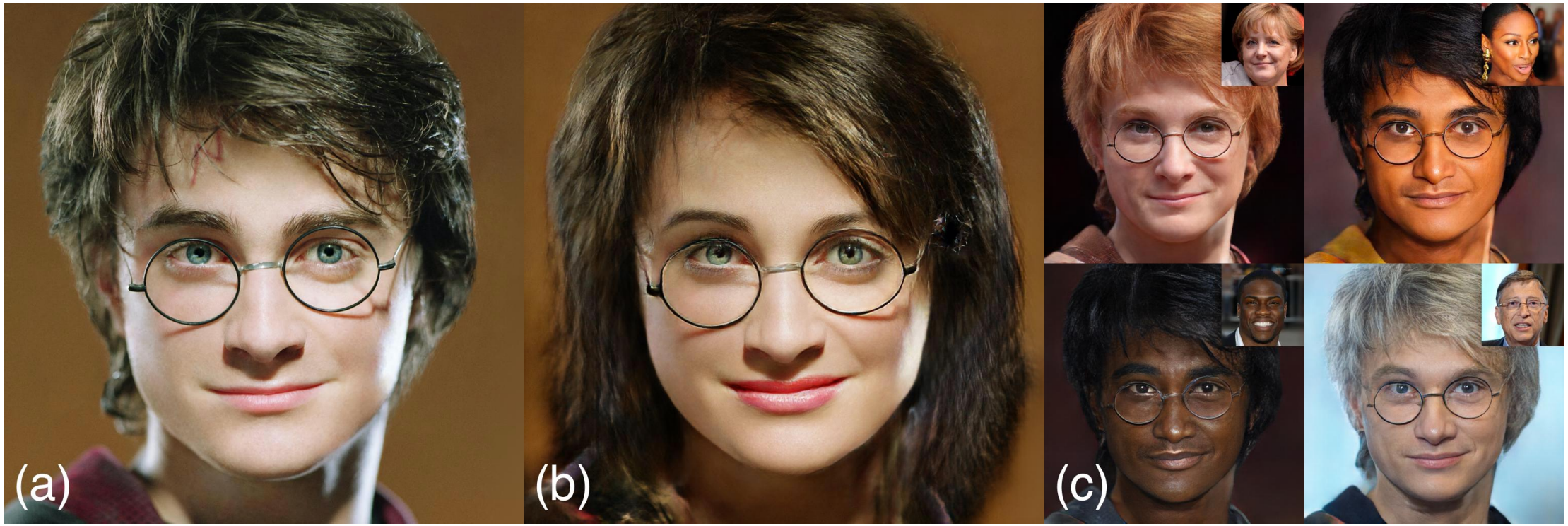


Image manipulation examples generated by our method from (a) source image sampled from Celeba-HQ: (b) gender swap at 1024x1024 and (c) style mixing at 512x512. Samples are generated feed-forward, StyleGANv2 which we distilled was trained on FFHQ

https://ar5iv.labs.arxiv.org/html/2003.03581

APPENDIX

FID

- **Measure Distance Between Distributions**
- Now we measure how far these two Gaussian distributions are using something called the **Fréchet distance**.
- it basically measures:
- Difference between means
- Difference between spread (covariance)
- The formula is:
- FID = $\left\|\mu_r - \mu_g\right\|^2 + Tr\left(\Sigma_\gamma + \Sigma_g - 2(\Sigma_r \Sigma g)^{\frac{1}{2}}\right)$
- But conceptually:
- If distributions are similar → small distance → low FID
- If distributions are very different → large distance → high FID

CYCLEGAN

- *Cycle-Consistent Generative Adversarial Network :* a framework designed for image-to-image translation tasks where paired examples are not available

Suppose you want to translate:

- 🐴 Horse → Zebra
- 🌞 Summer → ❄️ Winter
- 🎨 Monet painting → 📷 Real photo
- 📷 MRI → CT

❗ You **don't have paired images**.

For Example: Generating RGB imagery from SAR, multispectral imagery from RGB, map routes from satellite imagery.

Image A (horse) ↔ Exact same scene as zebra

You only have:

- A dataset of horses
- A dataset of zebras

No matching pairs.

👉 CycleGAN solves **unpaired image-to-image translation**.

That's its main purpose.

WHY NORMAL GAN IS NOT ENOUGH

- A normal GAN can:
  - Generate zebras
  - Generate horses
  - But it **cannot translate a specific horse into its zebra version**.

This exact horse → zebra version of this horse

That’s different from just generating random zebras.

THE BIG IDEA OF CYCLEGAN

- Instead of one GAN, CycleGAN uses:
- **TWO generators**
- G: X → Y (Horse → Zebra)
- F: Y → X (Zebra → Horse)
- **TWO discriminators**
- D_Y → judges if image is real zebra
- D_X → judges if image is real horse
- So total:
- 2 Generators
- 2 Discriminators

CYCLE CONSISTENCY

1. Take a horse
2. Convert it to zebra using G
3. Convert it back using F

- You should get back the original horse.
- The reconstructed horse should ≈ original horse.

TRAINING FLOW (STEP-BY-STEP)

- Generate fake zebra
  - y_fake = G(x)
  - Reconstruct
  - x_recon = F(y_fake)

- Compute:
- Compute loss
- Cycle loss between x and x_recon